Experiment No. 2
Implement a multi-armed bandit problem and compare different exploration strategies like epsilon-greedy and UCB.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: To implement a Multi-Armed Bandit (MAB) problem and compare the performance of different exploration strategies such as Epsilon-Greedy and Upper Confidence Bound (UCB) in maximizing cumulative reward.

Objective:
- To understand the concept of the **Multi-Armed Bandit problem** in reinforcement learning.
- To implement the **Epsilon-Greedy strategy** for balancing exploration and exploitation.
- To implement the **Upper Confidence Bound (UCB) algorithm** for action selection.
- To compare the performance of both strategies based on:

  - Cumulative reward
  - Regret (difference from optimal reward)
  - Convergence behavior

Theory:

The Multi-Armed Bandit Problem (MAB) is a fundamental problem in the field of reinforcement learning and decision-making under uncertainty. The problem involves a gambler who has to choose among several slot machines (also called "one-armed bandits") with unknown payout probabilities. The gambler's objective is to maximize his or her total payout by choosing the best slot machine to play.
The MAB problem is commonly used in various fields, such as clinical trials, online advertising, and recommendation systems. The MAB problem can be seen as a trade-off between exploration and exploitation. Exploration refers to the gambler trying out different slot machines to learn about their payout probabilities, while exploitation refers to the gambler sticking to the slot machine that has shown the highest payout probability so far.

A bandit is defined as someone who steals your money. A one-armed bandit is a simple slot machine wherein you insert a coin into the machine, pull a lever, and get an immediate reward. But why is it called a bandit? It turns out all casinos configure these slot machines in such a way that all gamblers end up losing money!

A multi-armed bandit is a complicated slot machine wherein instead of 1, there are several levers that a gambler can pull, with each lever giving a different return. The probability distribution for the reward corresponding to each lever is different and is unknown to the gambler

Multi-Armed Bandit Problem

The Multi-Armed Bandit problem is a classic problem in reinforcement learning where an agent must choose between multiple actions (arms), each providing a stochastic reward.

- Each arm has an **unknown probability distribution** of rewards.
- The goal is to **maximize total reward over time**.
- The key challenge is the **exploration vs exploitation trade-off**:
  - **Exploration**: Trying new arms to gain information
  - **Exploitation**: Choosing the best-known arm

Epsilon-Greedy Strategy

This is a simple and widely used method:

- With probability **ε (epsilon)** → choose a random action (**exploration**)
- With probability **1 − ε** → choose the best-known action (**exploitation**)

Advantages:

- Easy to implement
- Ensures continuous exploration

Disadvantages:

- May explore too much or too little depending on ε
- Fixed ε may not adapt over time

Upper Confidence Bound (UCB)

UCB is a more advanced strategy that balances exploration and exploitation using statistical confidence.

Idea:

- Choose the action with the highest **upper confidence bound** value
- It considers:
  - Average reward of the arm
  - Uncertainty (less tried arms get higher priority)

The selection formula is:

$$A_t = \arg\max_a \left(Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right)$$

Where:

- $Q_t(a)$: Estimated value of action a
- $N_t(a)$: Number of times action $a$is selected
- t: Total number of trials
- c: Exploration parameter

Advantages:

- Smart exploration based on uncertainty
- Better theoretical performance

Disadvantages:

- Slightly more complex to implement
- Requires careful tuning of parameter c

4. Performance Comparison

To evaluate both strategies:

- **Cumulative Reward**: Total reward collected over time
- **Regret**: Loss due to not always choosing the optimal arm
- **Convergence**: How quickly the algorithm identifies the best arm

Expected Outcome:

- Epsilon-Greedy is simpler but less efficient
- UCB generally achieves **better long-term performance**

Implementation:

import numpy as np

```
import matplotlib.pyplot as plt

# Step 1: Define Bandit Environment
n_arms = 5
true_probs = [0.1, 0.5, 0.3, 0.8, 0.6]  # true reward probabilities
n_steps = 1000

def pull_arm(arm):
    return 1 if np.random.rand() < true_probs[arm] else 0

# ------------------------------
# Epsilon-Greedy Implementation
# ------------------------------
epsilon = 0.1
Q_eps = np.zeros(n_arms)
N_eps = np.zeros(n_arms)
rewards_eps = []

for t in range(n_steps):
    if np.random.rand() < epsilon:
        action = np.random.randint(n_arms)
    else:
        action = np.argmax(Q_eps)

    reward = pull_arm(action)
    N_eps[action] += 1
    Q_eps[action] += (reward - Q_eps[action]) / N_eps[action]
    rewards_eps.append(reward)

# ------------------------------
# UCB Implementation
# ------------------------------
```

```
Q_ucb = np.zeros(n_arms)
N_ucb = np.zeros(n_arms)
rewards_ucb = []

c = 2

for t in range(n_steps):
    if t < n_arms:
        action = t
    else:
        ucb_values = Q_ucb + c * np.sqrt(np.log(t+1) / (N_ucb + 1e-5))
        action = np.argmax(ucb_values)

    reward = pull_arm(action)
    N_ucb[action] += 1
    Q_ucb[action] += (reward - Q_ucb[action]) / N_ucb[action]
    rewards_ucb.append(reward)

# Step 3: Compute cumulative rewards
cum_rewards_eps = np.cumsum(rewards_eps)
cum_rewards_ucb = np.cumsum(rewards_ucb)

# Step 4: Plot comparison
plt.plot(cum_rewards_eps, label="Epsilon-Greedy")
plt.plot(cum_rewards_ucb, label="UCB")
plt.xlabel("Steps")
plt.ylabel("Cumulative Reward")
plt.title("Epsilon-Greedy vs UCB")
plt.legend()
plt.show()
```

Conclusion:

The Multi-Armed Bandit problem demonstrates the importance of balancing exploration and exploitation.

- **Epsilon-Greedy** is simple and easy to implement but may explore inefficiently.
- **UCB** provides smarter exploration by considering uncertainty and performs better in the long run.
  Thus, UCB is generally more effective in maximizing cumulative reward.